

DEPARTMENT OF COMPUTER SCIENCE



University of
Reading

CS1IP: Imperative Programming

Week 10: Sorting

Learning Outcomes

- What is BubbleSort?
- What is MergeSort?
- Which are the Java Sorting functions?

What is a program?

A sequence of **instructions** that is **executed** by a **computer**.

Programs usually require handle data organized in a particular order.

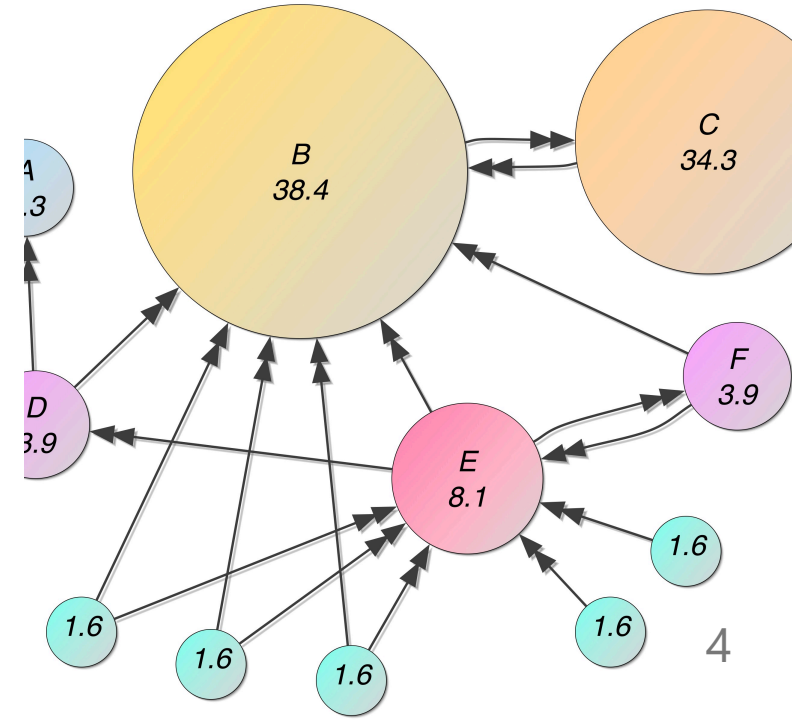
The make in such order, you need a sorting algorithm.

What is Sorting?

- It is the process of arranging data in a particular order (ascending or descending).
- Why Sorting Matters?
 - Enhances search performance.
 - Useful in applications like ranking, data organization, etc.

Case of Google Page Rank (1996)

- Larry Page and Sergey Brin while they were Ph.D. students at Stanford University
- PageRank Algorithm rank webpages given a user query.
 - It analyses link between webpages
 - Then assign a "score" to each webpage based on the q quality of links pointing to it.



Bubble Sort

Introduction to Bubble Sort

What is Bubble Sort?

It is a simple comparison-based sorting algorithm. Its name came from the idea of "**the biggest elements bubble up**".

Algorithm

1. **Iterate** through the list from the start to the end.
2. **Compare** adjacent elements.
3. **Swap** elements if they are out of order.
4. **Repeat** until the list is sorted or no swaps are needed.

i This can be done but the sorted to left of to the right. Here I will explain putting to the right



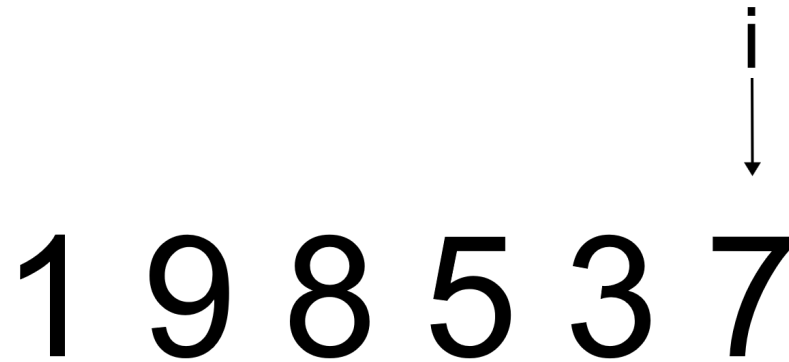
Bubble Sort: How it Works

- If the vector is already ordered in some iteration, it would be interesting to detect this and stop the algorithm.
- So, How to detect if a vector is ordered?

```
boolean sorted=true
....
    for (int j = 0; j < n-1; j++) {
        if (array[j] > array[j+1]) {
            ...
            sorted=false
        }
    }
```

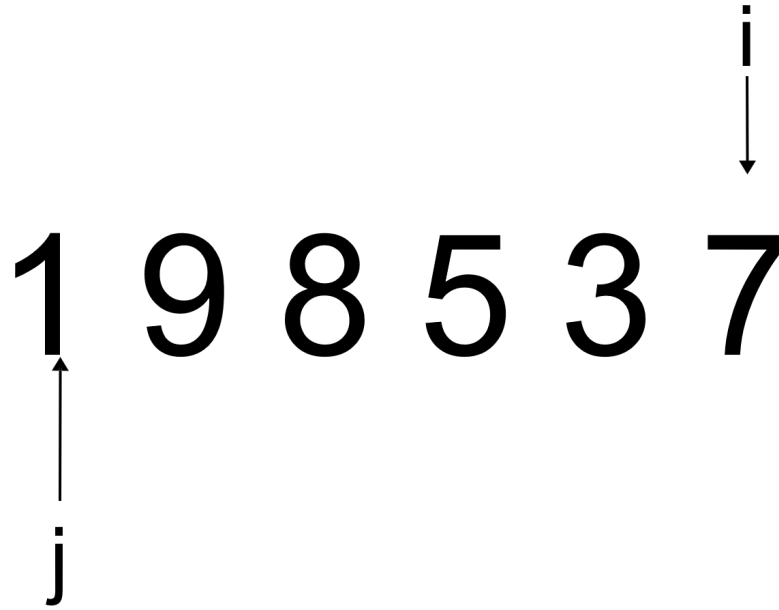
Bubble Sort: How it Works

1 9 8 5 3 7



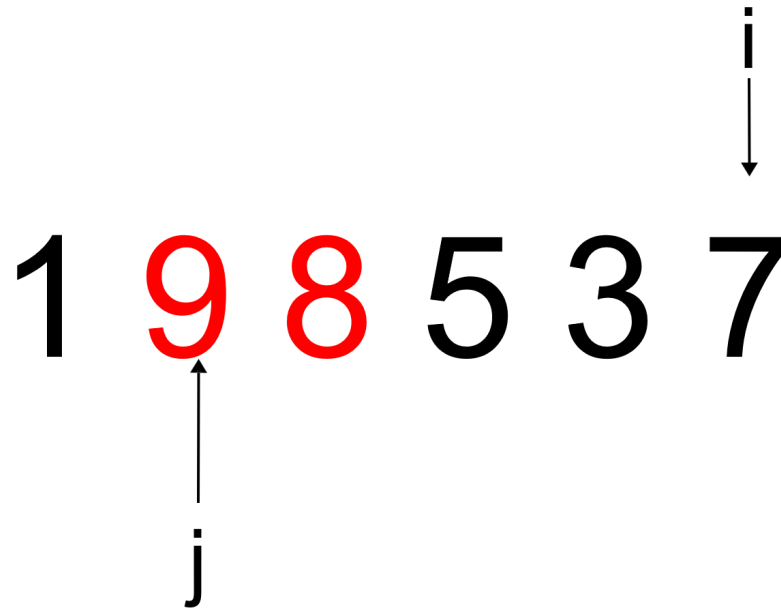
The idea now is to place the largest element at the end of the vector, comparing consecutive ones. The pointer i indicates this position

Bubble Sort: How it Works



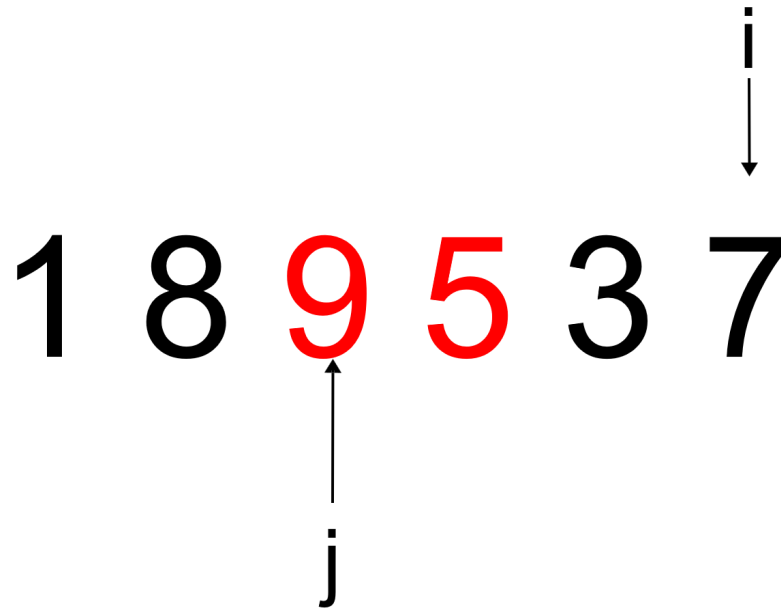
```
if (array[j] > array[j+1]) {  
    int temp = array[j];  
    array[j] = array[j+1];  
    array[j+1] = temp;  
}
```

Bubble Sort: How it Works



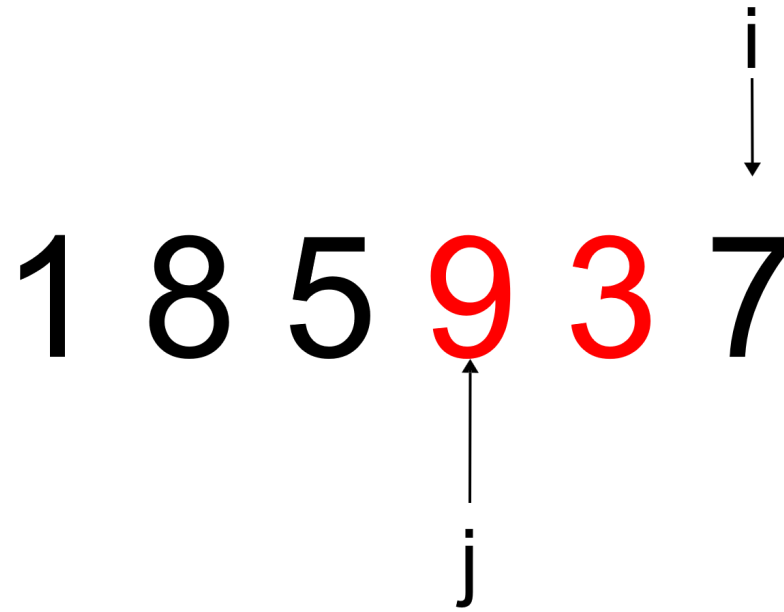
```
if (array[j] > array[j+1]) {  
    int temp = array[j];  
    array[j] = array[j+1];  
    array[j+1] = temp;  
}
```

Bubble Sort: How it Works



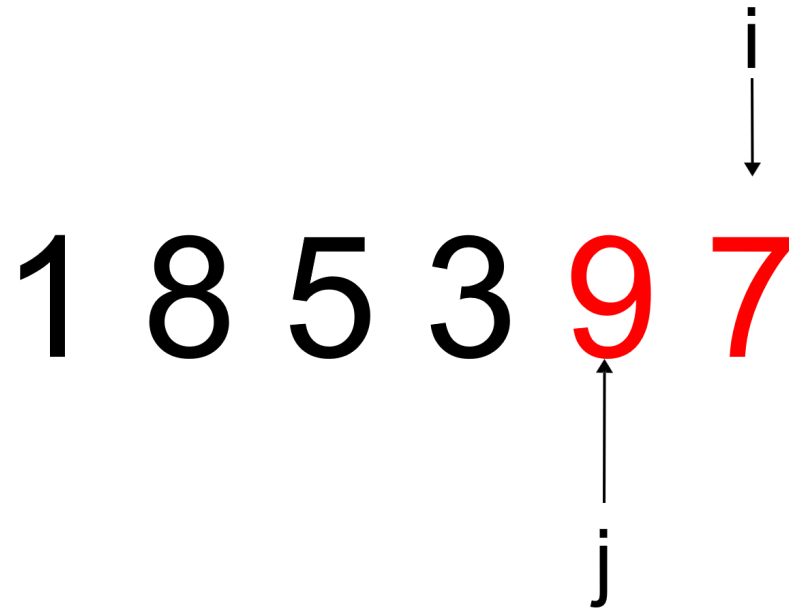
```
if (array[j] > array[j+1]) {  
    int temp = array[j];  
    array[j] = array[j+1];  
    array[j+1] = temp;  
}
```

Bubble Sort: How it Works



```
if (array[j] > array[j+1]) {  
    int temp = array[j];  
    array[j] = array[j+1];  
    array[j+1] = temp;  
}
```

Bubble Sort: How it Works

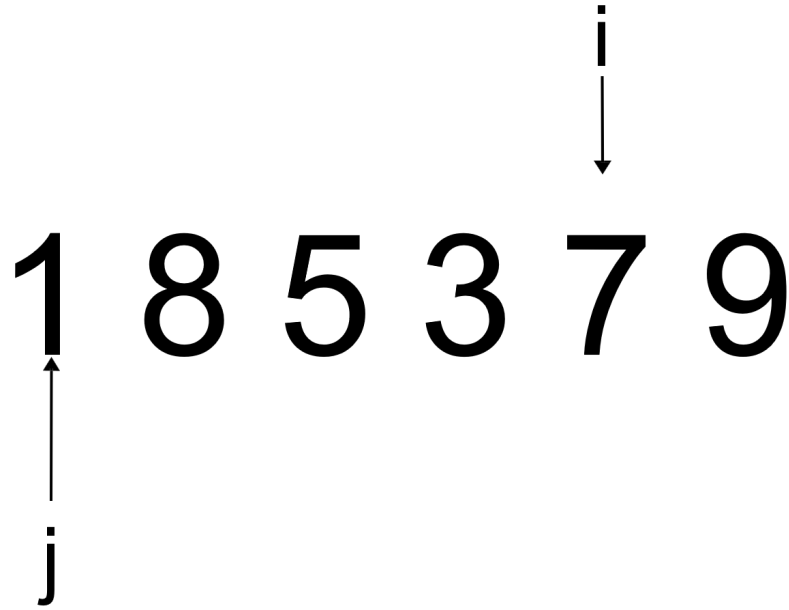


The vector to the right of i has its elements ordered, in the definitive position

Bubble Sort: How it Works

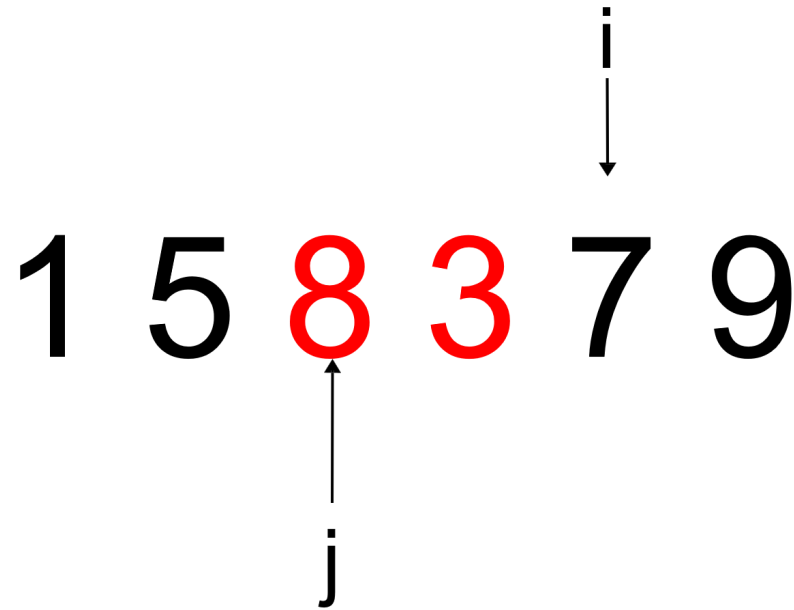
i
↓
1 8 5 3 7 9

Bubble Sort: How it Works



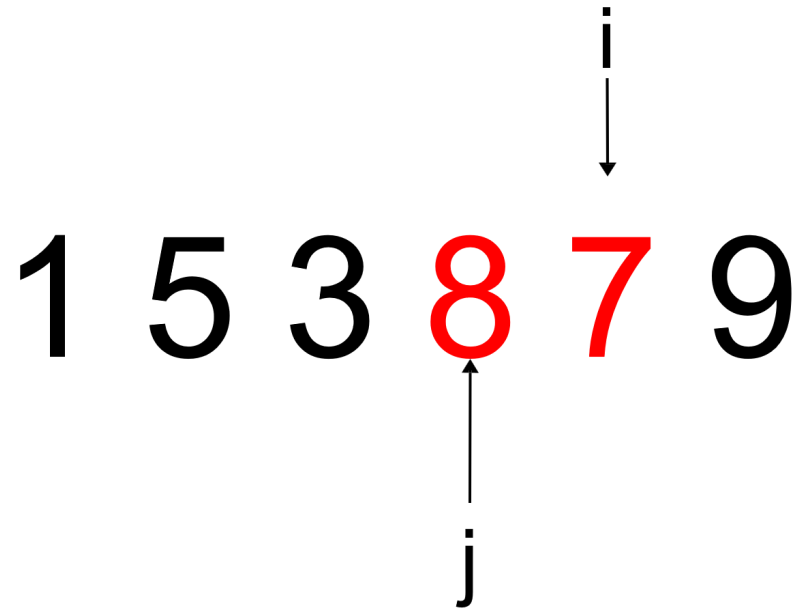
```
if (array[j] > array[j+1]) {  
    int temp = array[j];  
    array[j] = array[j+1];  
    array[j+1] = temp;  
}
```

Bubble Sort: How it Works



```
if (array[j] > array[j+1]) {  
    int temp = array[j];  
    array[j] = array[j+1];  
    array[j+1] = temp;  
}
```

Bubble Sort: How it Works



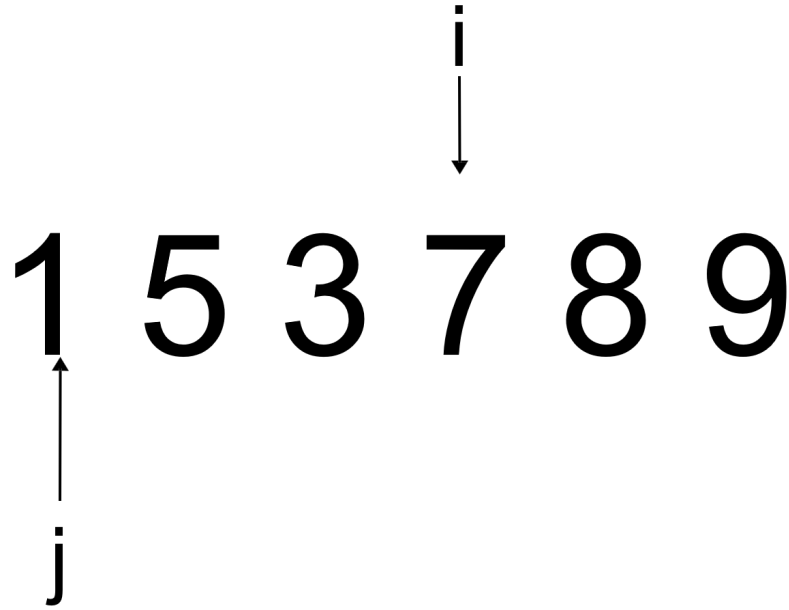
```
if (array[j] > array[j+1]) {  
    int temp = array[j];  
    array[j] = array[j+1];  
    array[j+1] = temp;  
}
```

Bubble Sort: How it Works

i
↓
1 5 3 7 8 9

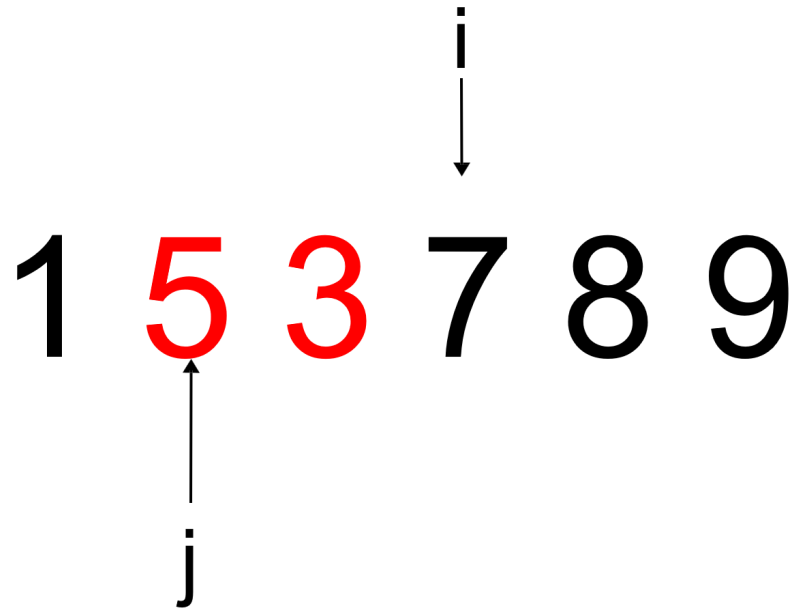
The vector to the right of i has its elements ordered, in the definitive position

Bubble Sort: How it Works



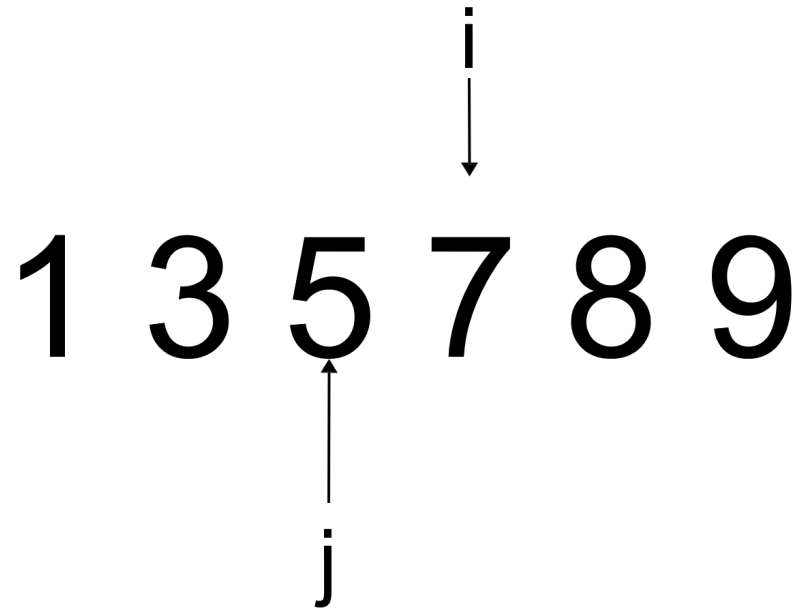
```
if (array[j] > array[j+1]) {  
    int temp = array[j];  
    array[j] = array[j+1];  
    array[j+1] = temp;  
}
```

Bubble Sort: How it Works



```
if (array[j] > array[j+1]) {  
    int temp = array[j];  
    array[j] = array[j+1];  
    array[j+1] = temp;  
}
```

Bubble Sort: How it Works



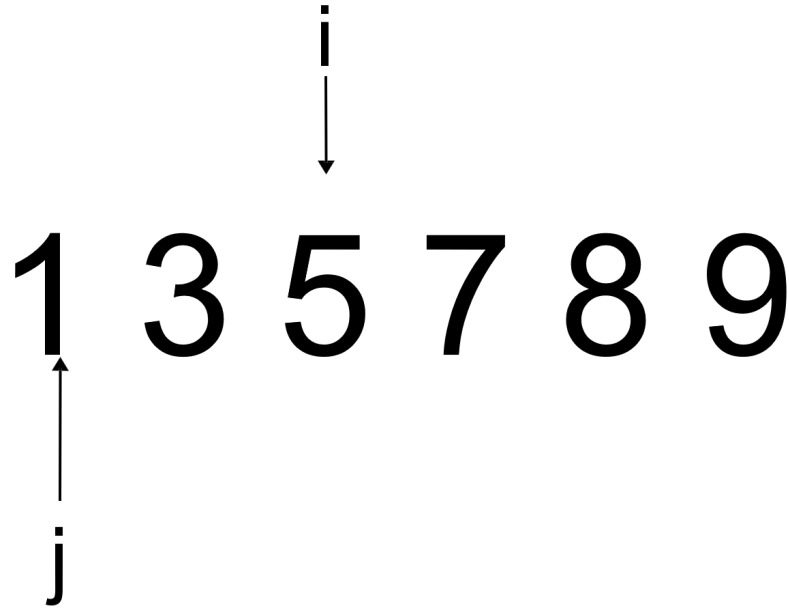
```
if (array[j] > array[j+1]) {  
    int temp = array[j];  
    array[j] = array[j+1];  
    array[j+1] = temp;  
}
```

Bubble Sort: How it Works

i
↓
1 3 5 7 8 9

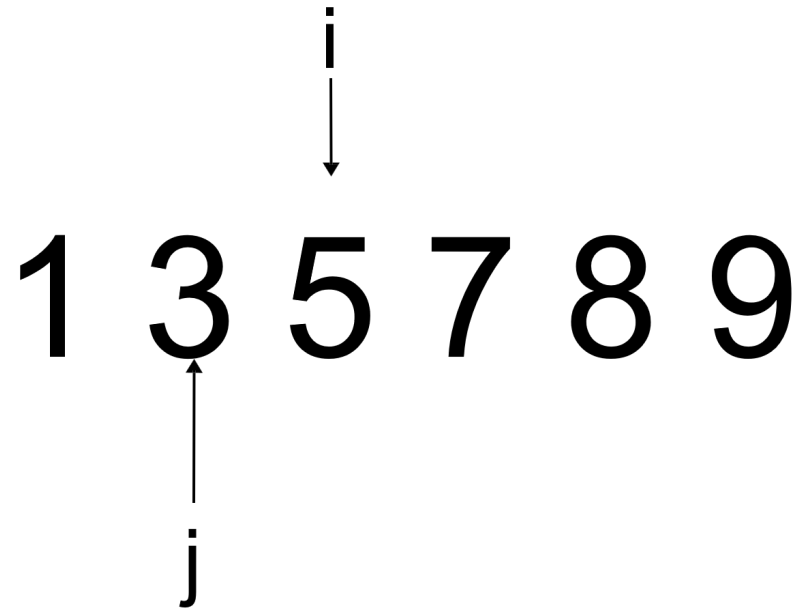
The vector to the right of i has its elements ordered, in the definitive position

Bubble Sort: How it Works



```
if (array[j] > array[j+1]) {  
    int temp = array[j];  
    array[j] = array[j+1];  
    array[j+1] = temp;  
}
```

Bubble Sort: How it Works



```
if (array[j] > array[j+1]) {  
    int temp = array[j];  
    array[j] = array[j+1];  
    array[j+1] = temp;  
}
```

Bubble Sort: How it Works

i
↓
1 3 5 7 8 9

There were no swaps! The vector is sorted

Bubble Sort in Java

```
int[] numbers = {1, 9, 8, 5, 3, 7};  
bubbleSort(numbers);  
System.out.println("Sorted array:");  
for (int num : numbers) {  
    System.out.print(num + " ");  
}
```

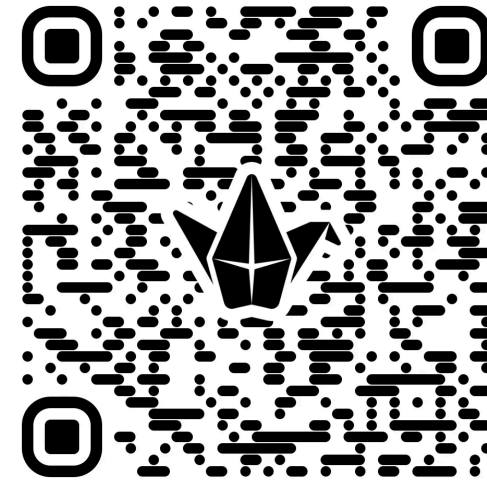
Bubble Sort in Java

```
void bubbleSort(int[] array) {  
    int n = array.length;  
    boolean sorted=true;  
    for (int i = n-1; i >=0 ; i--) {  
        for (int j = 0; j < n-1; j++) {  
            if (array[j] > array[j+1]) {  
                int temp = array[j];  
                array[j] = array[j+1];  
                array[j+1] = temp;  
                sorted=false;  
            }  
        }  
        if (sorted){  
            break;  
        }  
    }  
}
```

Quiz 1 ([click here](#) or scan QRCode)

What is the best-case time complexity of Bubble Sort?

- a. $O(n)$
- b. $O(n^2)$
- c. $O(n \log n)$
- d. $O(1)$



- **Best Case:** $O(n)$ (array is already sorted)
- **Worst Case:** $O(n^2)$

Merge Sort

What is Merge Sort?

It is a divide-and-conquer algorithm that splits the array into halves, sorts them, and then merges the sorted halves.

Key concepts

- **Definition:** A Divide and Conquer algorithm that splits the array into halves, sorts them, and then merges the sorted halves.
- **Efficiency:** $O(n \log n)$ time complexity
- **Stability:** Merge Sort is a stable sort algorithm.

Algorithm

Divide: Split the array into two halves

- in next, `mergeSort` function handle this split

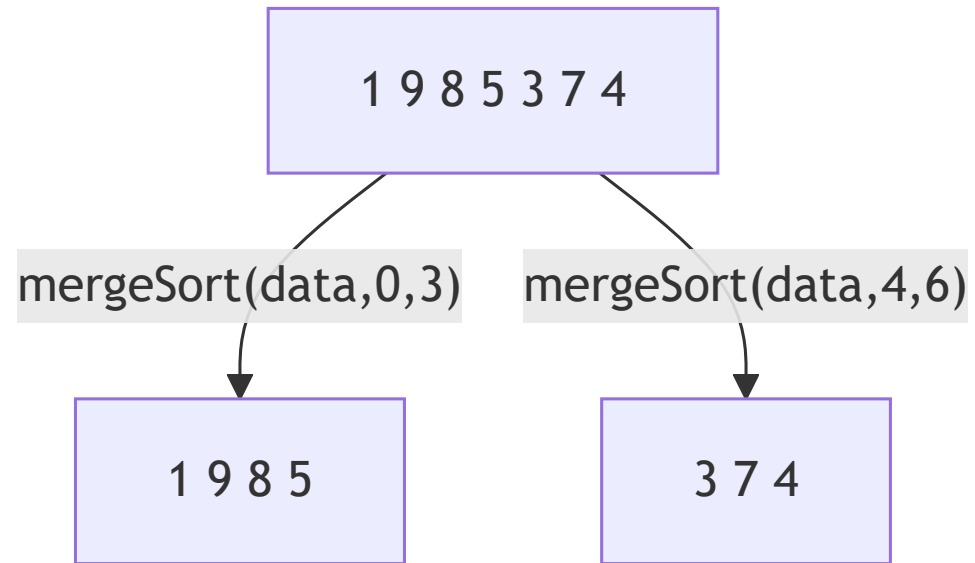
Combine: Merge the sorted halves to produce the final sorted array

- next `merge` function handle this combine two sorted sub-arrays into the original array.

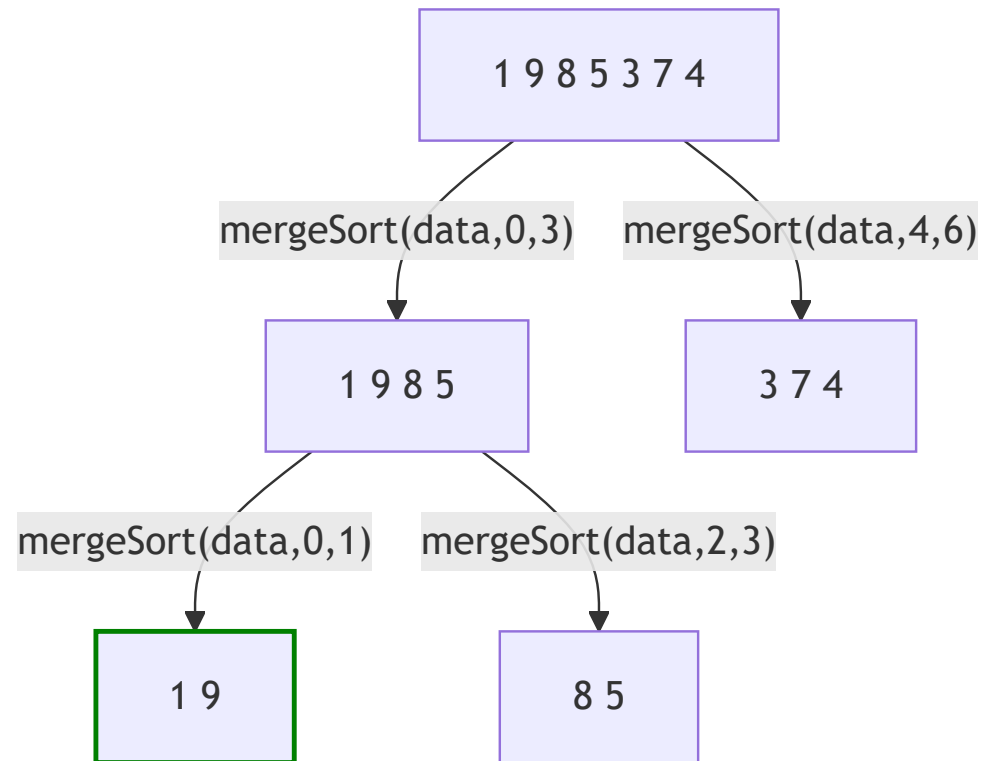
Merge Sort: How it Works

1 9 8 5 3 7 4

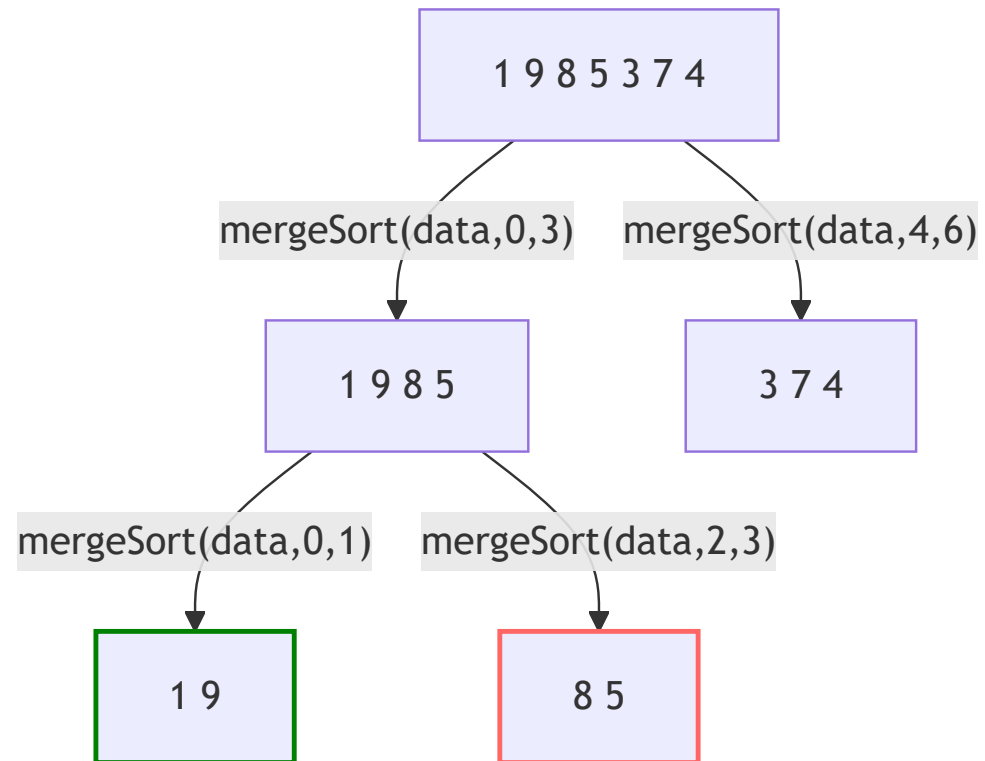
Merge Sort: How it Works



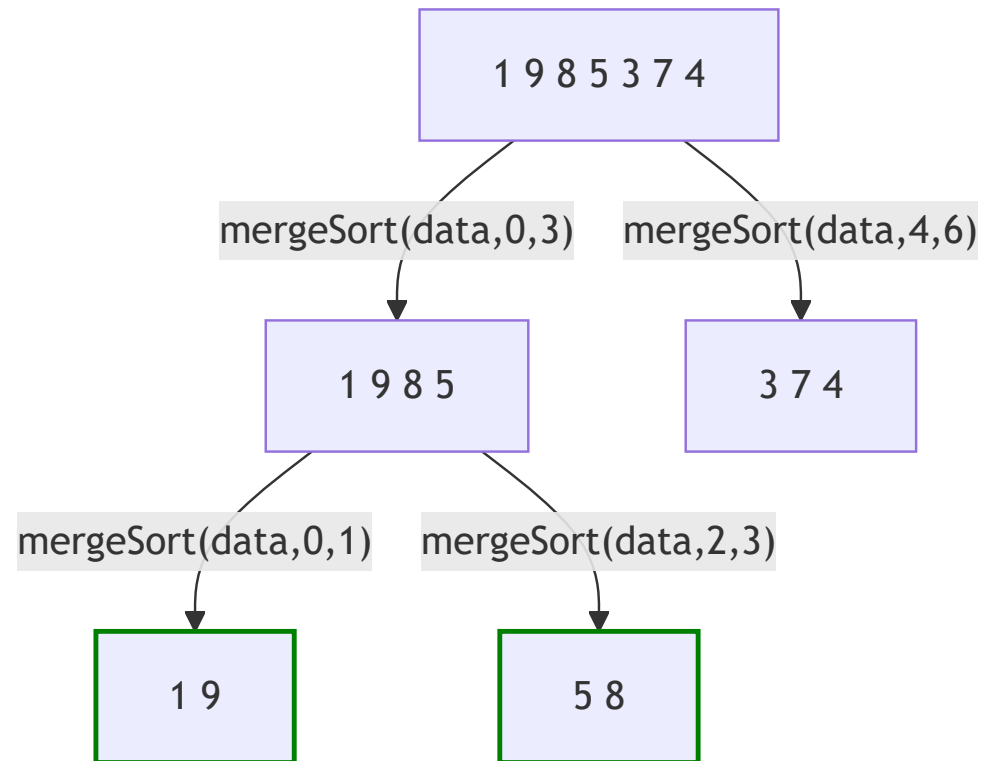
Merge Sort: How it Works



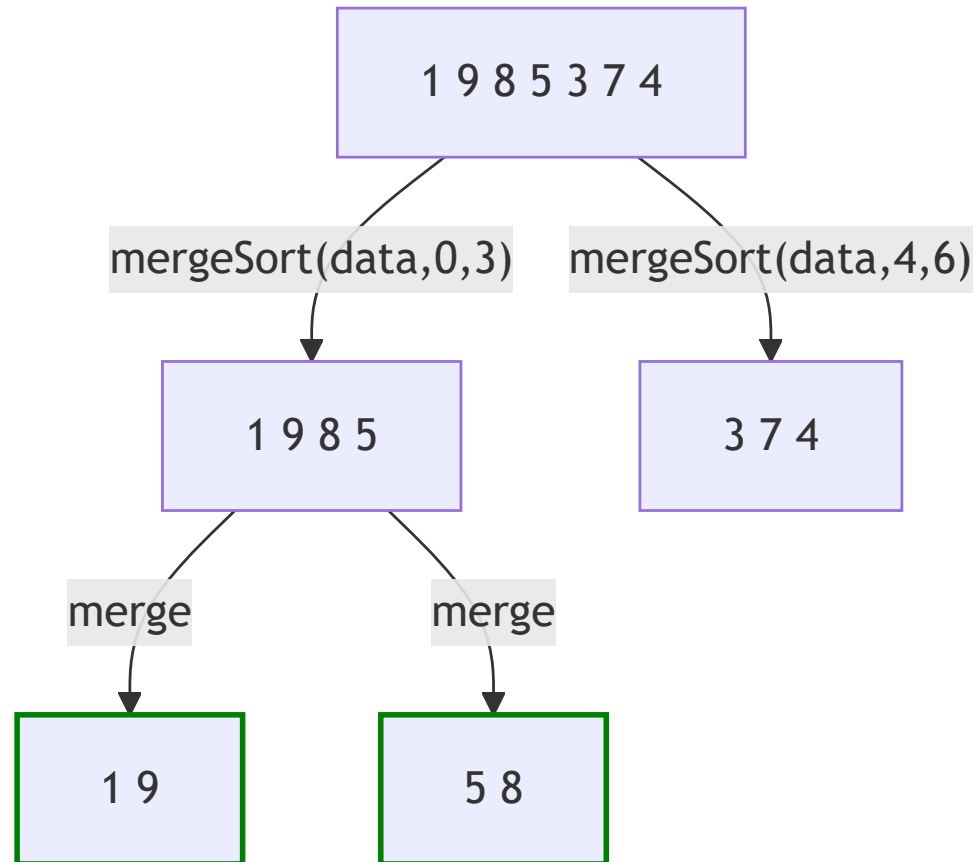
Merge Sort: How it Works



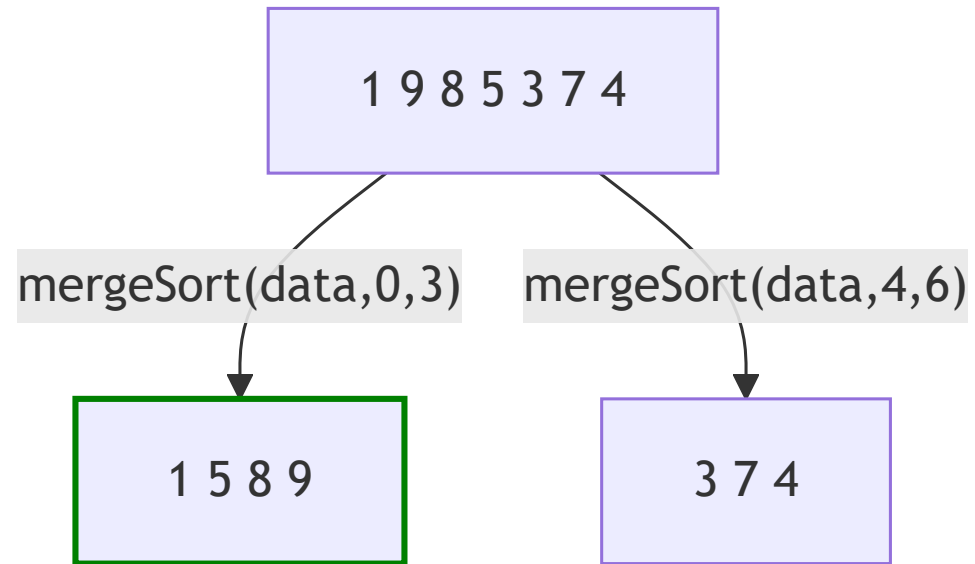
Merge Sort: How it Works



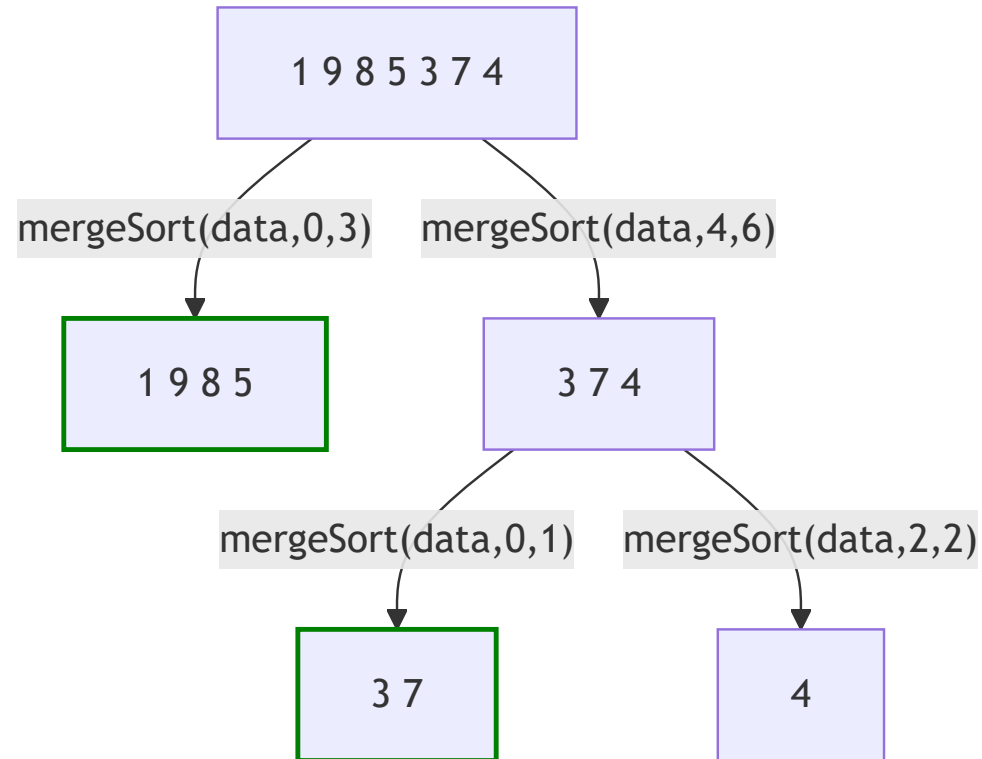
Merge Sort: How it Works



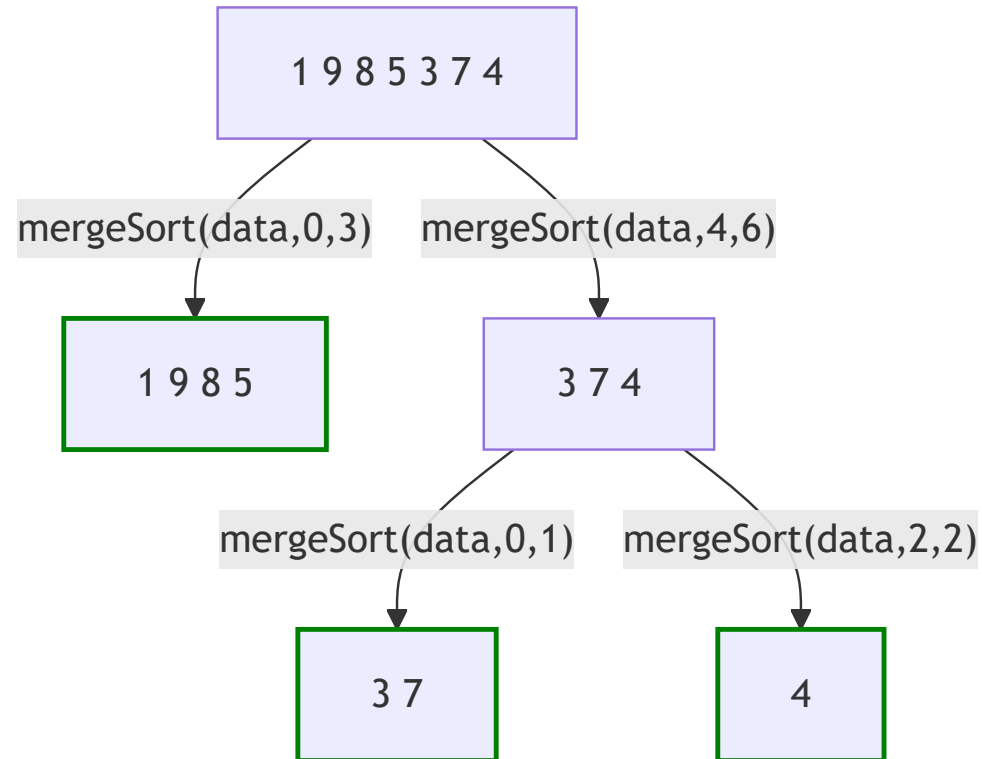
Merge Sort: How it Works



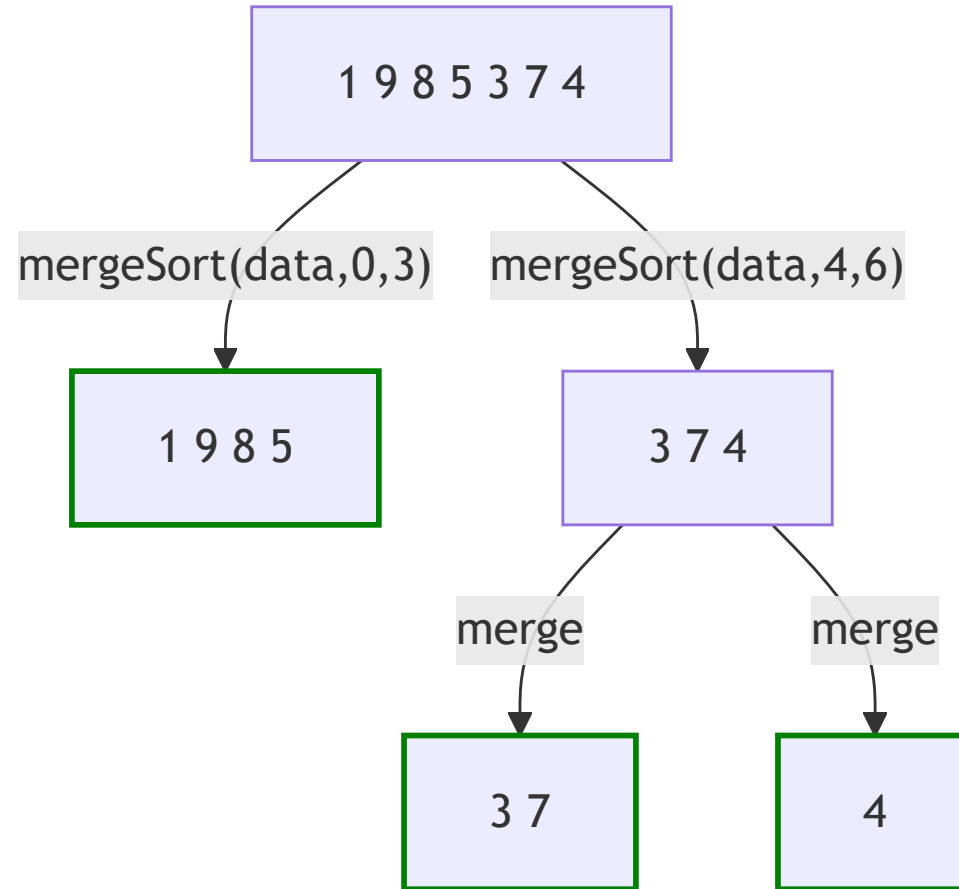
Merge Sort: How it Works



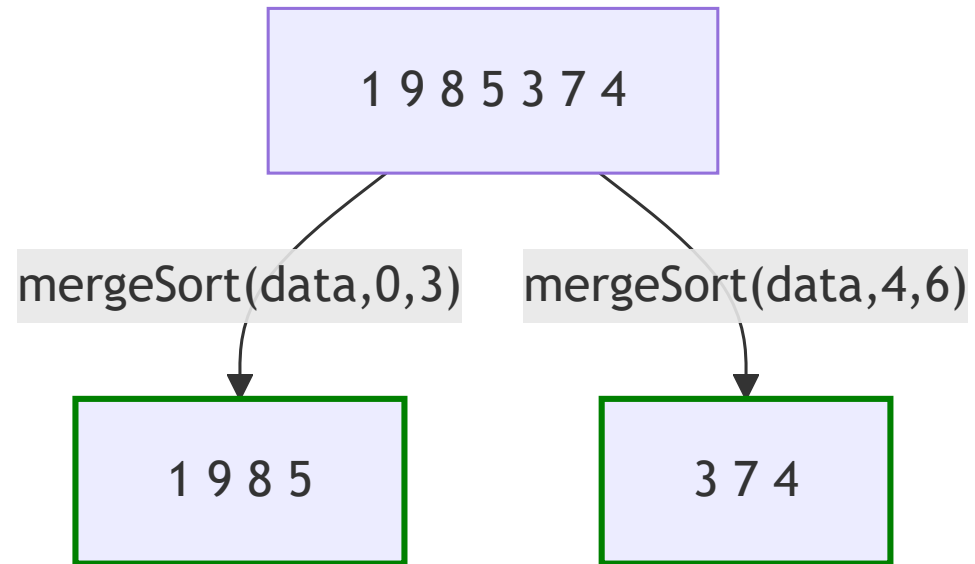
Merge Sort: How it Works



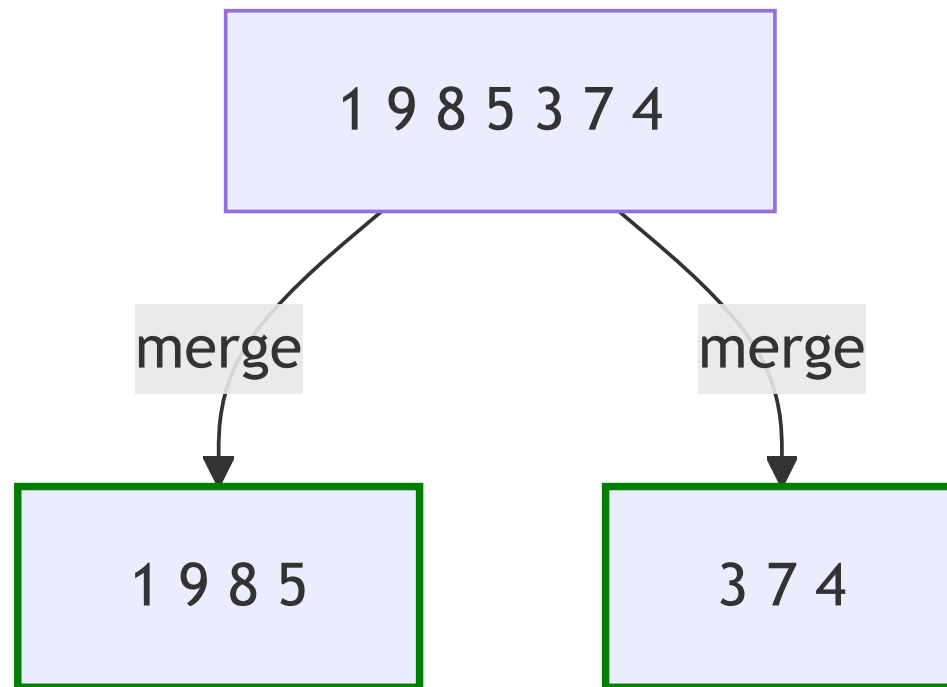
Merge Sort: How it Works



Merge Sort: How it Works



Merge Sort: How it Works



Merge Sort: How it Works



1 3 4 5 7 8 9

MergeSort in Java

```
void mergeSort(int[] data, int left, int right) {  
    ...  
}  
void merge(int[] data, int left, int middle1, int middle2, int right) {  
    ...  
}  
int[] numbers = {5, 3, 8, 4, 2};  
mergeSort(data, 0, numbers.length - 1); // sort entire array
```

- `mergeSort` handle sorting from left to right indexes.
- `merge` merges two sorted sub-arrays into the original array.

MergeSort in Java

```
void mergeSort(int[] data, int left, int right) {  
    if ((right - left) >= 1) {  
        int middle1 = (left + right) / 2;  
        int middle2 = middle1 + 1;  
        mergeSort(data, left, middle1);  
        mergeSort(data, middle2, right);  
        merge (data, left, middle1, middle2, right);  
    }  
}
```

- `mergeSort(data, left, middle1)` handle sorting the left part sub-array
- `mergeSort(data, middle2, right)` handle sorting of right sub-array.

```
void merge(int[] data, int left, int middle1, int middle2, int right) {  
    int leftIndex = left;  
    int rightIndex = middle2;  
    int combinedIndex = left;  
    int[] combined = new int[data.length];  
    // merge arrays until reaching end of either  
    while (leftIndex <= middle1 && rightIndex <= right) {  
        if (data[leftIndex] <= data[rightIndex]) {  
            combined[combinedIndex++] = data[leftIndex++];  
        }  
        else {  
            combined[combinedIndex++] = data[rightIndex++];  
        }  
    }  
    ...  
}
```

- **Two Pointers:** Used to compare elements in `leftIndex` and `rightIndex`.

```

void merge(int[] data, int left, int middle1, int middle2, int right) {
    ...
    // if left array is empty
    if (leftIndex == middle2) {
        while (rightIndex <= right) {
            combined[combinedIndex++] = data[rightIndex++];
        }
    }
    else { // right array is empty
        while (leftIndex <= middle1) {
            combined[combinedIndex++] = data[leftIndex++];
        }
    }

    // copy values back into original array
    for (int i = left; i <= right; i++) {
        data[i] = combined[i];
    }
}

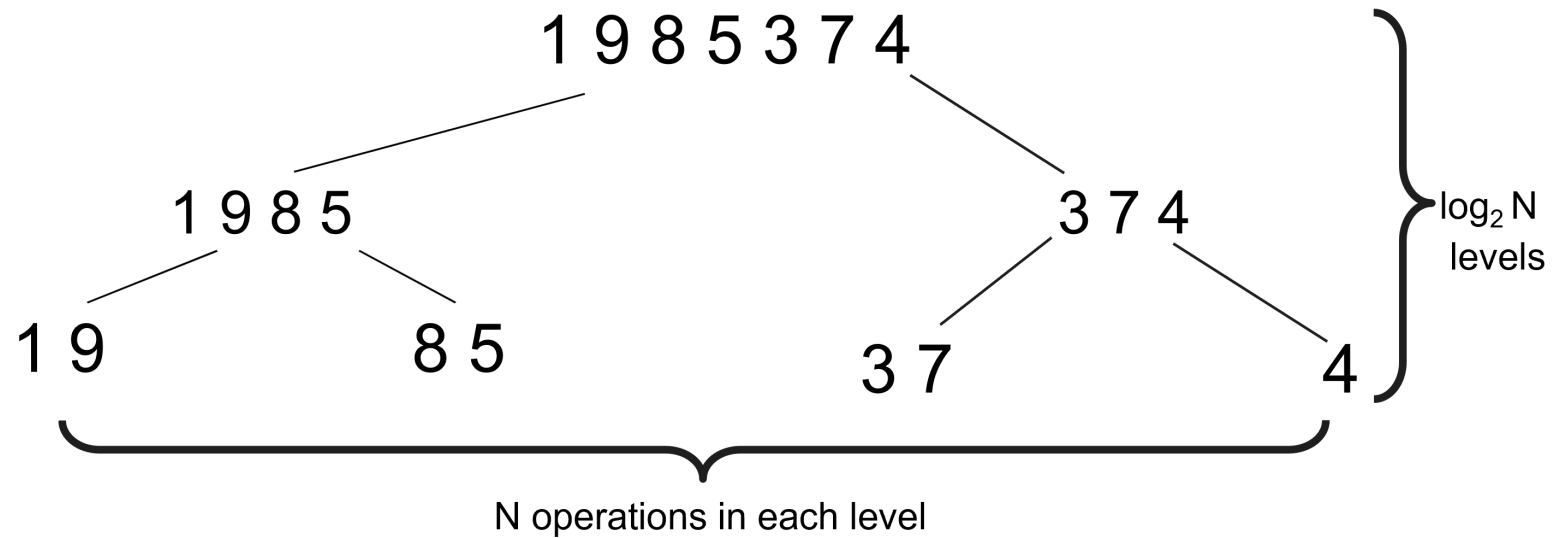
```

```
int[] numbers = {5, 3, 8, 4, 2};  
mergeSort(numbers);  
  
System.out.println("Sorted array:");  
for (int num : numbers) {  
    System.out.print(num + " ");  
}
```

Merge Sort Time Complexity

 Which is its Best Case and Worst Case complexity?

Merge Sort Time Complexity



Merge Sort Time Complexity

Best Case: $O(n \log n)$

- MergeSort always divides the array into halves and processes each half recursively
- run regardless of how the data is already sorted.
- The merging process, which combines two sorted halves into one, takes linear time $O(n)$ for each level of recursion.

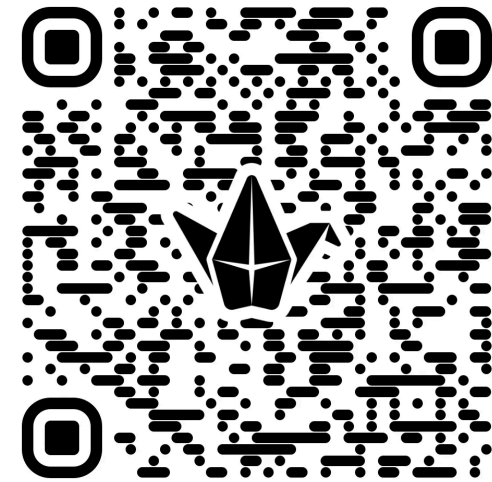
Worst Case: $O(n \log n)$

- MergeSort has the same time complexity in the worst case as in the best case, because the divide-and-merge process remains consistent regardless of the initial order of the input.

Quiz 2 ([click here](#) or scan QRCode)

Why is Merge Sort considered more efficient than Bubble Sort for large datasets?

- a. Merge Sort uses logarithmic space
- b. Merge Sort has a time complexity of $O(n^2)$
- c. Merge Sort divides the problem into smaller sub-problems and combines solutions
- d. Bubble Sort performs fewer swaps



Comparing BubbleSort and MergeSort

Comparing BubbleSort and MergeSort

Why compare sorting algorithms?

- Understand performance differences
- Discuss BubbleSort's simplicity vs. MergeSort's efficiency
 - **BubbleSort:** $O(n^2)$
 - **MergeSort:** $O(n \log n)$

Comparing BubbleSort and MergeSort

```
generateSortInput(String filename, int count) throws IOException {  
    Random rand = new Random();  
    FileWriter writer = new FileWriter(filename);  
    for (int i = 0; i < count; i++) {  
        writer.write(rand.nextInt(10000) + "\n");  
    }  
    writer.close();  
}  
generateSortInput(10, "sort10.txt");  
generateSortInput(100, "sort100.txt");  
generateSortInput(10000, "sort10000.txt");
```

Measure Bubble Sort and Merge Sort

Consider that we have `measureMergeSort` and `measureBubbleSort`

```
long measureMergeSort(String filename) throws IOException {  
    ...  
}  
long measureBubbleSort(String filename) throws IOException {  
    ...  
}
```

Measure Bubble Sort and Merge Sort

```
System.out.println("Bubble Sort:")
measureMergeSort("input10.txt");
measureMergeSort("input100.txt");
measureMergeSort("input10000.txt");
System.out.println("Merge Sort:")
measureBubbleSort("input10.txt");
measureBubbleSort("input100.txt");
measureBubbleSort("input10000.txt");
```

Bubble Sort:

0

3

1589

Merge Sort:

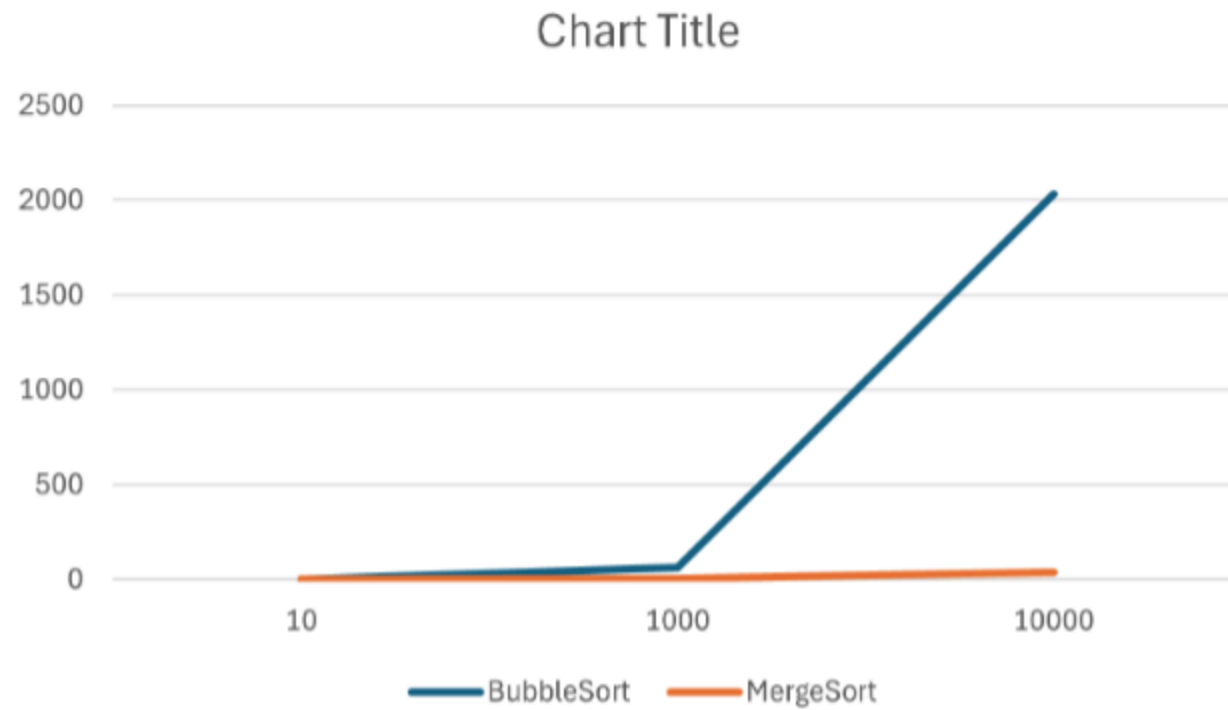
1

2

20

Comparing plot

As expect:



What else about sort?

There is other Sorting methods that you will see DataStructure and Algorithms

- MergeSort
- InsertSort
- RadixSort
- QuickSort

 But should we implement sorting for every time?

- No! There are already Sorting functions

Java Sorting functions

Introduction to Sorting in Java

- Sorting in Java:
 - Built-in sorting functions in Java for ease of use.
 - `Arrays.sort()` : Sorts arrays.
 - `Collections.sort()` : Sorts Collections.
- Key Features:
 - Highly optimized.
 - Supports natural and custom orders.
 - Handles int and object arrays.

Sorting Arrays with Java functions

- Using `Arrays.sort()` for int array

```
int[] numbers = {5, 3, 8, 1};  
Arrays.sort(numbers);  
System.out.println(Arrays.toString(numbers)); // [1, 3, 5, 8]
```

- Using `Arrays.sort()` for String array

```
String[] names = { "Bob", "Alice", "Charlie"};  
Arrays.sort(names);  
System.out.println(Arrays.toString(names)); // [Alice, Bob, Charlie]
```

Sorting Collections with Java functions

- Using `Collections.sort()` for ArrayList

```
var numbers = new ArrayList<Integer>(List.of(5, 3, 8, 1));  
Collections.sort(numbers);  
System.out.println(numbers); // [1, 3, 5, 8]
```

- Using `Arrays.sort()` for ArrayList

```
var names = new ArrayList<String>(List.of("Alice", "Bob", "Charlie"));  
Arrays.sort(names);  
System.out.println(names); // [Alice, Bob, Charlie]
```

 But if we need a custom type of comparison between the elements?

Custom comparison with Java Comparator

- Comparator allows defining custom sorting logic directly within the `Arrays.sort` or `Collections.sort`
 - See Documentation of [Arrays.sort](#) and [Collections.sort](#)
- i** The Comparator is usually requires a Java Class, but here we used it simplified way called **inline Comparator**

Java inline Comparator - Syntax

Replace `TYPE` for type of element to be compared (e.g. `Integer`, `String`):

For Arrays:

```
Arrays.sort(names, new Comparator<TYPE>() {  
    public int compare(TYPE o1, TYPE o2) {  
        // Define custom comparison logic here  
    }  
});
```

For Collections:

```
Collections.sort(list, new Comparator<TYPE>() {  
    public int compare(TYPE o1, TYPE o2) {  
        // Define custom comparison logic here  
    }  
});
```

Java inline Comparator - Syntax

For Arrays:

```
String[] names = { "Bob", "Alice", "Charlie"};
Arrays.sort(names, new Comparator<String>() {
    public int compare(String o1, String o2) {
        return o1.compareTo(o2);
    }
});
System.out.println(Arrays.toString(names));
```

For Collections:

```
var names = new ArrayList<String>(List.of("Alice", "Bob", "Charlie"));
Collections.sort(names, new Comparator<String>() {
    public int compare(String o1, String o2) {
        return o1.compareTo(o2);
    }
});
```

Comparator for sorting Numbers in Acceding Order

- Using a Comparator Function

```
var numbers = new LinkedList<Integer>(Arrays.asList(42, 5, 16, 8));
Collections.sort(numbers, new Comparator<Integer>() {
    public int compare(Integer a, Integer b) {
        return a - b;
    }
});
System.out.println(numbers); // [5, 8, 16, 42]
```

Comparator for Sorting Numbers in Descending Order

- Using a Comparator Function

```
var numbers = new LinkedList<Integer>(Arrays.asList(42, 5, 16, 8));
Collections.sort(numbers, new Comparator<Integer>() {
    public int compare(Integer a, Integer b) {
        return b - a;
    }
});
System.out.println(numbers); // [42, 16, 8, 5]
```

Comparator Sorting Strings by Length

```
var words = new LinkedList<String>(Arrays.asList("apple", "banana", "kiwi"));
Collections.sort(words, new Comparator<String>() {
    public int compare(String a, String b) {
        return a.length() - b.length();
    }
});
System.out.println(words); // [kiwi, apple, banana]
```

Comparator Example: Sorting Student IDs

- Student IDs Example
- Format: Two uppercase letters followed by six digits (e.g., "AB123456").

```
List<String> studentIds = new ArrayList<>();  
studentIds.add("AB123456");  
studentIds.add("CD654321");  
studentIds.add("EF234567");  
  
Collections.sort(studentIds);  
System.out.println(studentIds); // [AB123456, CD654321, EF234567]
```

Comparator Example: Sorting Student IDs

Numerically only

```
Collections.sort(studentIds, new Comparator<String>() {  
    public int compare(String id1, String id2) {  
        int num1 = Integer.parseInt(id1.substring(2));  
        int num2 = Integer.parseInt(id2.substring(2));  
        return num1 - num2;  
    }  
});  
System.out.println(studentIds); // [AB123456, EF234567, CD654321]
```

Comparator Example: Sorting Student IDs

Letters only

```
Collections.sort(studentIds, new Comparator<String>() {  
    public int compare(String id1, String id2) {  
        String letters1 = id1.substring(0, 2);  
        String letters2 = id2.substring(0, 2);  
        return letters1.compareTo(letters2);  
    }  
});  
System.out.println(studentIds); // [AB123456, CD654321, EF234567]
```

Advanced: Parallel Sorting

Using Arrays.parallelSort

```
int[] largeArray = {5, 2, 8, 1, 3};  
Arrays.parallelSort(largeArray);  
System.out.println(Arrays.toString(largeArray)); // [1, 2, 3, 5, 8]
```

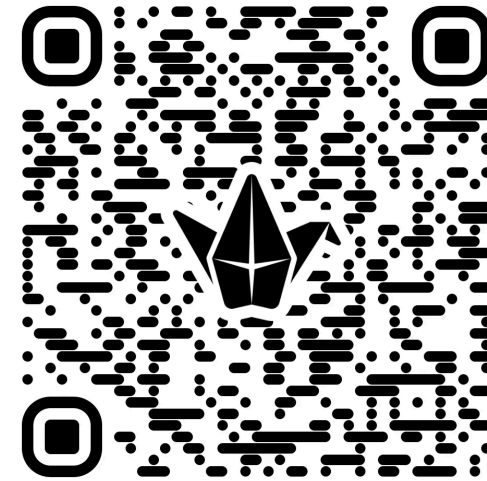
When to Use Parallel Sorting?

- For large datasets where multi-processor can boost performance.
- Less efficient for small datasets.

Quiz 2 ([click here](#) or scan QRCode)

What is the primary advantage of using a Comparator in Java?

- a. It simplifies the sorting process for primitive types
- b. It allows for custom sorting logic for non-primitive types
- c. It improves the performance of Bubble Sort
- d. It replaces the need for Merge Sort



Summary of Java Sorting functions

- The main functions are: `Arrays.sort()` and `Collections.sort()`
- Performance:
 - Optimized sorting algorithms.
- Ease of Use:
 - Minimal code required for common use cases.
- Flexibility:
 - Support primitive types (e.g. `int`) and objects (e.g. `String`)
 - Comparing elements can be extended using a Comparator

Review Learning Outcomes

Review Learning Outcomes

What is BubleSort?

- Simple and inefficient comparison-based sorting algorithm.

What is MergeSort?

- Algorithm efficient for sorting large arrays

Which are the Java Sorting functions?

- `sort()` is a function at each Collection. If you are using non-basic type, you can use a Comparator

DEPARTMENT OF COMPUTER SCIENCE



Thank you! 😊

Do you have any questions 🙋 ?

The next lecture will cover: More collections and Streams